



INSTITUTO POLITECNICO NACIONAL

**UNIDAD PROFESIONAL INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS AVANZADAS**

“Práctica 4: Problemas de concurrencia”

Peralta Ordoñez Jesús

Valencia Cedeño Marcos Gael

Vargas Clemente Leonel Zajid

Asignatura: Bases de Datos Distribuidas

Profesor: De la Cruz Sosa Carlos

Grupo: 3TM2

Lectura Sucia / Dirty Read

Este problema ocurre porque leer datos sin usar bloqueos puede producir resultados inesperados. Otras conexiones pueden modificar datos de forma temporal, y si otra conexión utiliza esos nuevos datos no confirmados en sus cálculos, se llegará a resultados incorrectos. A esto se le conoce como leer "datos sucios" (dirty data), ya que los cálculos terminan dependiendo de información que aún no ha sido confirmada de manera definitiva en la base de datos.

Visualización antes de ejecutar los procesos

	DepartmentID	Name	GroupName	ModifiedDate
1	1	Engineering	Research and Development	2008-04-30 00:00:00.000

Proceso A (Conexión 1)

```
-- Proceso A
USE AdventureWorks2022;
GO

SELECT TOP 10 * FROM HumanResources.Department
where Name = 'Engineering'

-- 1. Iniciamos una transacción abierta
BEGIN TRANSACTION LecturaSucia;
UPDATE HumanResources.Department
SET GroupName = 'DirtyRead'
WHERE Name = 'Engineering';

PRINT 'Proceso A: Registro actualizado. Esperando antes de hacer COMMIT/ROLLBACK...';

-- 3. Pausamos la ejecución por 15 segundos para simular procesamiento
WAITFOR DELAY '00:00:8';

-- 4. Simulamos una cancelación o error, revirtiendo el cambio
ROLLBACK TRANSACTION;
PRINT 'Proceso A: Transacción revertida (ROLLBACK ejecutado).';
GO

SELECT TOP 10 * FROM HumanResources.Department
where Name = 'Engineering'
```

Proceso B (Conexión 2 - Ejecutar inmediatamente después de iniciar el Proceso A)

```
1      USE AdventureWorks2022;
2      GO
3
4      -- 1. Configuramos el nivel de aislamiento para permitir lecturas sucias
5      SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
6
7      PRINT 'Proceso B: Leyendo datos no confirmados...';
8
9      -- 2. Consultamos el mismo registro que el Proceso A está modificando
10     SELECT TOP 10 * FROM HumanResources.Department
11     where Name = 'Engineering'
12
13     -- 1. Configuramos el nivel de aislamiento para permitir lecturas sucias
14     SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
15
16     PRINT 'Proceso B: Leyendo datos no confirmados...';
17
```

Visualización de lo que sucedió

Resultado Proceso A

	DepartmentID	Name	GroupName	ModifiedDate
1	1	Engineering	Research and Development	2008-04-30 00:00:00.000
	DepartmentID	Name	GroupName	ModifiedDate
1	1	Engineering	Research and Development	2008-04-30 00:00:00.000

Resultado Proceso B

	DepartmentID	Name	GroupName	ModifiedDate
1	1	Engineering	DirtyRead	2008-04-30 00:00:00.000

Código de Solución (Nivel READ COMMITTED y operación atómica)

```
-- 1. Configuramos el nivel de aislamiento para permitir lecturas sucias
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

PRINT 'Proceso B: Leyendo datos no confirmados...';

-- 2. Consultamos el mismo registro que el Proceso A está modificando
SELECT TOP 10 * FROM HumanResources.Department
where Name = 'Engineering'

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Conclusiones del análisis

Al permitir explícitamente el nivel de aislamiento READ UNCOMMITTED, vulneramos la integridad de los datos, ya que permitimos que el sistema lea un registro que estaba "en el aire" dentro de una transacción abierta. Cuando esa transacción decidió hacer un ROLLBACK, nuestro Proceso B se quedó trabajando con un valor fantasma que ya no existe en la base de datos y lo guardó de forma permanente en otra tabla.

Al utilizar el nivel de aislamiento predeterminado READ COMMITTED, SQL Server asegura que otras conexiones no puedan ver los cambios antes de que sean considerados permanentes. Bajo esta configuración correcta, el Proceso B debe esperar forzosamente a que el Proceso A libere sus bloqueos (ya sea mediante un COMMIT o un ROLLBACK) antes de poder proceder con la lectura y la operación de actualización o inserción.

Análisis Inconsistente (Lectura No Repetible / Nonrepeatable Read)

Descripción del problema

En un entorno multiusuario, otros usuarios modifican dinámicamente los datos almacenados en una base de datos. Intentar ejecutar un proceso de larga duración, puede producir inconsistencias debido a los cambios que ocurren en la base de datos desde el principio hasta el final del reporte. Esto se considera un análisis inconsistente porque cada vez que se leen los datos, la información es diferente. A esta situación se le llama lecturas no repetibles.

Visualización antes de ejecutar los procesos

	ProductID	StartDate	EndDate	StandardCost	ModifiedDate
1	707	2011-05-31 00:00:00.000	2012-05-29 00:00:00.000	333.33	2012-05-29 00:00:00.000
2	707	2012-05-30 00:00:00.000	2013-05-29 00:00:00.000	333.33	2013-05-29 00:00:00.000
3	707	2013-05-30 00:00:00.000	NULL	333.33	2013-05-16 00:00:00.000

Proceso A (Conexión 1)

```
1      Use AdventureWorks2022
2      go
3
4      SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
5
6      begin transaction LecturasNoRepetidas;
7
8      PRINT 'Proceso A: Primera Lectura...';
9
10     select * from Production.ProductCostHistory
11     where ProductID = 707
12
13     -- Pausamos la ejecución por 15 segundos para simular procesamiento
14     WAITFOR DELAY '00:00:6';
15
16     PRINT 'Proceso A: Segunda lectura';
17
18     select * from Production.ProductCostHistory
19     where ProductID = 707
20
21     COMMIT TRANSACTION;
22     GO
```

Proceso B (Conexión 2 - Ejecutar justo después del Proceso A)

```
1      USE AdventureWorks2022;  
2      GO  
3  
4      UPDATE Production.ProductCostHistory  
5      SET StandardCost = StandardCost + $50  
6      WHERE ProductID = 707;  
7  
8      PRINT 'Proceso B: Cambio confirmado.';  
9      GO
```

Visualización de lo que sucedió

Resultado Proceso A

	ProductID	StartDate	EndDate	StandardCost	ModifiedDate
1	707	2011-05-31 00:00:00.000	2012-05-29 00:00:00.000	383.33	2012-05-29 00:00:00.000
2	707	2012-05-30 00:00:00.000	2013-05-29 00:00:00.000	383.33	2013-05-29 00:00:00.000
3	707	2013-05-30 00:00:00.000	NULL	383.33	2013-05-16 00:00:00.000
	ProductID	StartDate	EndDate	StandardCost	ModifiedDate
1	707	2011-05-31 00:00:00.000	2012-05-29 00:00:00.000	433.33	2012-05-29 00:00:00.000
2	707	2012-05-30 00:00:00.000	2013-05-29 00:00:00.000	433.33	2013-05-29 00:00:00.000
3	707	2013-05-30 00:00:00.000	NULL	433.33	2013-05-16 00:00:00.000

Resultado Proceso B

(3 filas afectadas)
Proceso B: Cambio confirmado.

Hora de finalización: 2026-09-24T09:05:25.1569706-06:00

Código de Solución ()

```
Use AdventureWorks2022|
go

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

begin transaction LecturasNoRepetidas;

PRINT 'Proceso A: Primera Lectura...';

select * from Production.ProductCostHistory
where ProductID = 707

-- Pausamos la ejecución por 15 segundos para simular procesamiento
WAITFOR DELAY '00:00:6';

PRINT 'Proceso A: Segunda lectura';

select * from Production.ProductCostHistory
where ProductID = 707

COMMIT TRANSACTION;
GO
```

Conclusiones del análisis

El análisis inconsistente ocurre al separar la lógica de un reporte en múltiples consultas independientes a lo largo del tiempo. Los cálculos resumidos en la Conexión A resultan ser incorrectos debido a las modificaciones realizadas simultáneamente por la Conexión B.

Para solucionar esto, además de empaquetar las consultas en una sola instrucción atómica, también se puede utilizar el nivel de aislamiento REPEATABLE READ. Al utilizar el nivel de aislamiento REPEATABLE READ, los datos recuperados quedan bloqueados hasta que la transacción termina, lo que previene que otras conexiones realicen cambios en esos datos.

Lecturas Fantasma (Phantom Reads)

Descripción del problema

Si otras conexiones insertan datos en el rango de los registros que estás analizando, puedes encontrarte con que esas nuevas filas aparecen en tus conjuntos de resultados sin motivo aparente desde el punto de vista de tu conexión. A estas nuevas filas se les llama *fantasmas* (phantoms). Este problema también produce análisis inconsistentes, ya que los totales previos dejan de ser válidos tras la inserción de las nuevas filas

Visualización antes de los procesos

	ProductSubcategoryID	ProductCategoryID	Name	rowguid	ModifiedDate
1	1	1	Mountain Bikes	2D364ADE-264A-433C-B092-4FCBF3804E01	2008-04-30 00:00:00.000
2	2	1	Road Bikes	000310C0-BCC8-42C4-B0C3-45AE611AF06B	2008-04-30 00:00:00.000
3	3	1	Touring Bikes	02C5061D-ECDC-4274-B5F1-E91D76BC3F37	2008-04-30 00:00:00.000
4	38	1	Lectura Fantasma	F32EC077-E63B-4E97-90D3-43C6AB4E3079	2026-09-24 02:17:30.017
5	40	1	Lectura Fantasma12	283760E6-B4A2-4B78-A5B7-734E23D91D96	2026-09-24 02:19:35.750
6	41	1	Lectura Fantasma121	8556DCA8-02BF-421E-AA95-A65344B95BBC	2026-09-24 02:20:17.853

Proceso A (Conexión 1)

```
Use AdventureWorks2022
go

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

begin transaction LecturasFantasmas;

PRINT 'Proceso A: Primera Lectura...';

select * from Production.ProductSubcategory
where ProductCategoryID = 1

-- Pausamos la ejecución por 15 segundos para simular procesamiento
WAITFOR DELAY '00:00:10';

PRINT 'Proceso A: Segunda lectura';

select * from Production.ProductSubcategory
where ProductCategoryID = 1

COMMIT TRANSACTION;
GO
```

Proceso B (Conexión 2 - Ejecutar inmediatamente después de iniciar el Proceso A)

```
Use AdventureWorks2022
go

insert into Production.ProductSubcategory(ProductCategoryID, Name, rowguid, ModifiedDate)
values (1, 'Lectura Fantasma125', NEWID(), GETDATE());

Print 'Fila insertada desde el proceso B'
```

Visualización de lo que sucedió

La visualización del error ocurre dentro de la misma ventana del Proceso A, ya que la segunda tabla resultado mostrará una fila más de la que debería

	ProductSubcategoryID	ProductCategoryID	Name	rowguid	ModifiedDate	
1	1	1	Mountain Bikes	2D364ADE-264A-433C-B092-4FCBF3804E01	2008-04-30 00:00:00.000	
2	2	1	Road Bikes	000310C0-BCC8-42C4-B0C3-45AE611AF06B	2008-04-30 00:00:00.000	
3	3	1	Touring Bikes	02C5061D-ECDC-4274-B5F1-E91D76BC3F37	2008-04-30 00:00:00.000	
4	38	1	Lectura Fantasma	F32EC077-E63B-4E97-90D3-43C6AB4E3079	2026-09-24 02:17:30.017	
5	40	1	Lectura Fantasma12	283760E6-B4A2-4B78-A5B7-734E23D91D96	2026-09-24 02:19:35.750	
6	41	1	Lectura Fantasma121	8556DCA8-02BF-421E-AA95-A65344B95BBC	2026-09-24 02:20:17.853	

	ProductSubcategoryID	ProductCategoryID	Name	rowguid	ModifiedDate	
1	1	1	Mountain Bikes	2D364ADE-264A-433C-B092-4FCBF3804E01	2008-04-30 00:00:00.000	
2	2	1	Road Bikes	000310C0-BCC8-42C4-B0C3-45AE611AF06B	2008-04-30 00:00:00.000	
3	3	1	Touring Bikes	02C5061D-ECDC-4274-B5F1-E91D76BC3F37	2008-04-30 00:00:00.000	
4	38	1	Lectura Fantasma	F32EC077-E63B-4E97-90D3-43C6AB4E3079	2026-09-24 02:17:30.017	
5	40	1	Lectura Fantas...	283760E6-B4A2-4B78-A5B7-734E23D91D96	2026-09-24 02:19:35.750	
6	41	1	Lectura Fantas...	8556DCA8-02BF-421E-AA95-A65344B95BBC	2026-09-24 02:20:17.853	
7	42	1	Lectura Fantas...	22AD7BF1-210D-4C82-8CE0-B3C5250F3699	2026-09-24 09:12:23.240	

Código de Solución

Prevenir este problema es más difícil que en los casos anteriores, porque SQL Server necesita bloquear filas que aún no existen (bloquear datos inexistentes). Para solucionarlo, se puede utilizar el nivel de aislamiento SERIALIZABLE o la sugerencia de optimizador (optimizer hint) SERIALIZABLE.

```
Use AdventureWorks2022
go
```

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
begin transaction LecturasFantasmas;
```

```
PRINT 'Proceso A: Primera Lectura...';
```

```
select * from Production.ProductSubcategory
where ProductCategoryID = 1
```

```
-- Pausamos la ejecución por 15 segundos para simular procesamiento
WAITFOR DELAY '00:00:10';
```

```
PRINT 'Proceso A: Segunda lectura';
```

```
select * from Production.ProductSubcategory
where ProductCategoryID = 1
```

```
COMMIT TRANSACTION;
GO
```

	ProductSubcategoryID	ProductCategoryID	Name	rowguid	ModifiedDate
1	1	1	Mountain Bikes	2D364ADE-264A-433C-B092-4FCBF3804E01	2008-04-30 00:00:00.000
2	2	1	Road Bikes	000310C0-BCC8-42C4-B0C3-45AE611AF06B	2008-04-30 00:00:00.000
3	3	1	Touring Bikes	02C5061D-ECDC-4274-B5F1-E91D76BC3F37	2008-04-30 00:00:00.000
4	38	1	Lectura Fantasma	F32EC077-E63B-4E97-90D3-43C6AB4E3079	2026-09-24 02:17:30.017
5	40	1	Lectura Fantasma12	283760E6-B4A2-4B78-A5B7-734E23D91D96	2026-09-24 02:19:35.750
6	41	1	Lectura Fantasma121	8556DCA8-02BF-421E-AA95-A65344B95BBC	2026-09-24 02:20:17.853
7	42	1	Lectura Fantasma125	22AD7BF1-210D-4C82-8CE0-B3C5250F3699	2026-09-24 09:12:23.240

	ProductSubcategoryID	ProductCategoryID	Name	rowguid	ModifiedDate
1	1	1	Mountain Bikes	2D364ADE-264A-433C-B092-4FCBF3804E01	2008-04-30 00:00:00.000
2	2	1	Road Bikes	000310C0-BCC8-42C4-B0C3-45AE611AF06B	2008-04-30 00:00:00.000
3	3	1	Touring Bikes	02C5061D-ECDC-4274-B5F1-E91D76BC3F37	2008-04-30 00:00:00.000
4	38	1	Lectura Fantasma	F32EC077-E63B-4E97-90D3-43C6AB4E3079	2026-09-24 02:17:30.017
5	40	1	Lectura Fantasma12	283760E6-B4A2-4B78-A5B7-734E23D91D96	2026-09-24 02:19:35.750
6	41	1	Lectura Fantasma...	8556DCA8-02BF-421E-AA95-A65344B95BBC	2026-09-24 02:20:17.853
7	42	1	Lectura Fantasma...	22AD7BF1-210D-4C82-8CE0-B3C5250F3699	2026-09-24 09:12:23.240

Conclusiones del análisis

Tener lecturas fantasmas no es algo que deba evitarse por sí solo, a menos que produzcan análisis inconsistentes, lo cual suele ser el caso habitual. Al presentarse inserciones de otras conexiones en rangos de datos que estamos leyendo, nuestras segundas lecturas arrojarán cantidades distintas. Implementar el nivel `SERIALIZABLE` soluciona el problema al colocar bloqueos de rango de claves, garantizando que ninguna otra transacción pueda insertar filas en el rango que nuestra consulta abarca hasta que terminemos de procesar